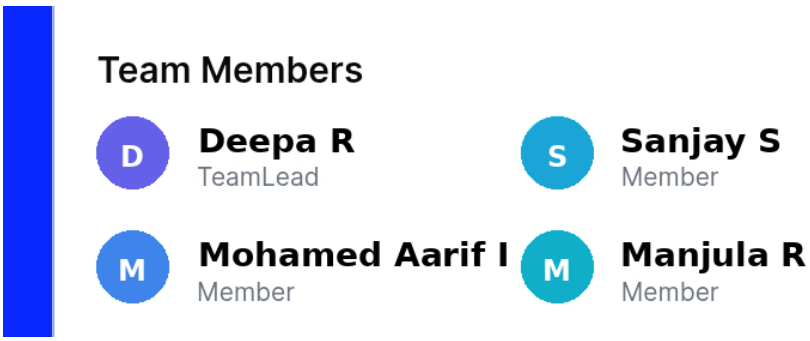
PocketSmart AI

Smart Budget Planning with AI

Team Members:-

1. Team Lead - Deepa R
2. Member 1 - Sanjay S
3. Member 2 - Mohamed Aarif I
4. Member 3 - Manjula R



Technical Architecture:-

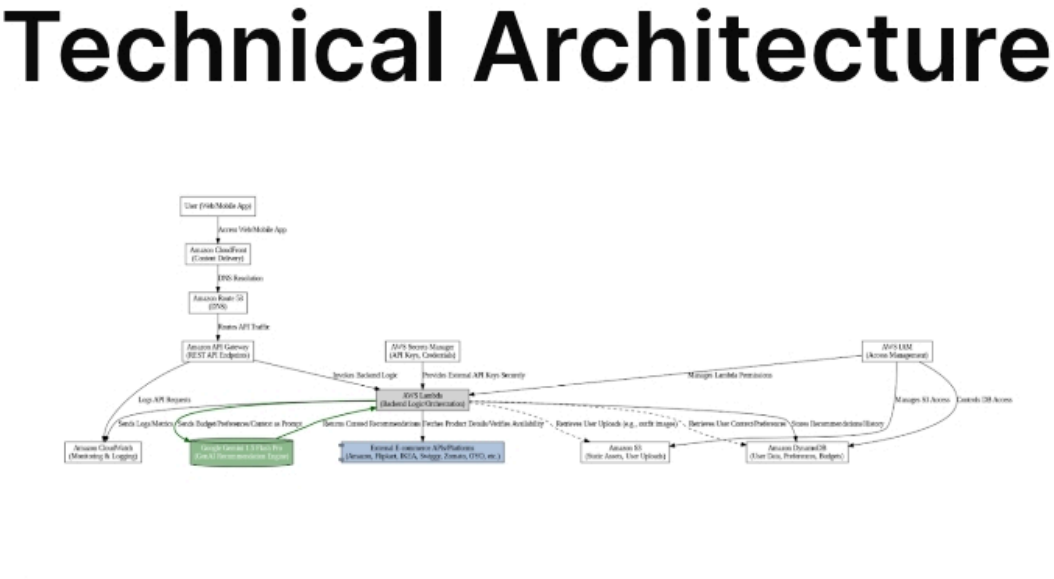


Figure 1: PocketSmart AI – Technical Architecture (AWS)

Edit with WPS Office

Introduction:-

Planning how to spend money on a room makeover, a family party or a jewelry purchase is a common problem. People often overspend on one item and run short on another because they have no clear split to follow. PocketSmart AI is a smart budget planning assistant designed to solve this problem. It is developed using HTML, CSS and JavaScript, and its planned architecture uses AWS services with Google Gemini as the AI engine. The user chooses a category, enters a budget in rupees (₹) and instantly receives a clear, item-wise budget plan on a mobile-friendly screen.

Objective of the project:-

The primary objectives of our project are:

- To provide an instant and easy-to-read budget split for common spending needs.
- To reduce the time and confusion involved in planning a budget manually.
- To create a simple, user-friendly and mobile-responsive planning tool.
- To build a base that can be extended with Generative AI (Gemini) product recommendations.
- To help users spend smartly across Home Interior, Party Planning and Jewelry.

System requirements:-

Hardware Requirements:-

- Device: Smartphone, laptop or desktop computer
- RAM: 2 GB minimum
- Storage: 50 MB free space
- Internet: Required for opening the deployed application

Software Requirements:-

- Operating System: Windows 10 / Android
- Front-end: HTML5, CSS3
- Scripting Language: JavaScript
- IDE: Visual Studio Code
- Browser: Google Chrome
- Cloud (planned): AWS Lambda, API Gateway, DynamoDB, S3, CloudFront
- AI (planned): Google Gemini API


Edit with WPS Office

Existing System:-

In the existing system, people plan budgets using notebooks, calculators or spreadsheets, or they ask friends and shop owners for advice. Each person makes their own split, which is often based on guesswork.

The drawbacks are:

- Time-consuming manual calculation.
- No standard guidance on how much to spend on each item.
- High chance of overspending on one item.
- No smart or personalised suggestions.
- Spreadsheets are difficult to use on a mobile phone.

Proposed System:-

To overcome these drawbacks, we propose PocketSmart AI. Our system gives a ready-made budget split for the chosen category and is designed to connect to Google Gemini for AI recommendations. The working steps are:

- User selects a category (Home Interior, Party Planning or Jewelry Recommendation).
- A second option appears for that category (Room Type, Party Type or Jewelry Type).
- User enters the budget amount in rupees.
- On clicking Plan My Budget, the input is validated.
- The percentage split for that category is applied to the budget.
- The plan is displayed on the screen within a second.

Advantages:-

Fast, Simple, User-Friendly, Mobile Responsive, Free to use and 24/7 Availability.

System Architecture:-

The working prototype follows a simple client-side flow:

User Interface (HTML/CSS) -> JavaScript Logic (Validation and Calculation) -> Budget Plan Displayed on UI.

The planned production architecture is shown in Figure 1. The user reaches the app through Amazon CloudFront and Route 53. Requests pass through Amazon API Gateway to AWS Lambda, which acts as the backend logic and orchestration layer. Lambda talks to Google Gemini for AI recommendations, to external e-commerce APIs for product details and availability, to Amazon DynamoDB for user data, preferences

Edit with WPS Office

and budgets, and to Amazon S3 for static assets and user uploads. AWS Secrets Manager stores API keys, AWS IAM manages permissions and Amazon CloudWatch handles monitoring and logging.

Modules Description:-

1. **User Interface Module:** This is the front-end where users choose a category, enter the budget and view the plan. It is designed with clean HTML and CSS.

2. **Category Module:** Shows the extra dropdown (Room Type, Party Type or Jewelry Type) based on the selected category.

3. **Validation Module:** Checks that a category is selected and that the budget is a valid positive number.

4. **Budget Calculation Module:** Applies the percentage split of the chosen category to the entered budget.

5. **Response Display Module:** Formats the result with rupee symbols and Indian number format and shows it neatly to the user.

Edit with WPS Office

Category Module:-

When the user changes the category, the script clears the extra fields and adds the correct dropdown. For Home Interior it shows Room Type (Living Room, Bedroom, Kitchen). For Party Planning it shows Party Type (Birthday Party, Wedding, Family Function). For Jewelry Recommendation it shows Jewelry Type (Necklace, Earrings, Bangles, Ring).

```
// Show extra fields based on category
category.addEventListener("change", function () {
  extraFields.innerHTML = "";

  if (category.value === "home") {
    extraFields.innerHTML = `
      <label>Room Type:</label>
      <select id="roomType">
        <option>Living Room</option>
        <option>Bedroom</option>
        <option>Kitchen</option>
      </select>
    `;
  } else if (category.value === "party") {
    extraFields.innerHTML = `
      <label>Party Type:</label>
      <select id="partyType">
        <option>Birthday Party</option>
        <option>Wedding</option>
        <option>Family Function</option>
      </select>
    `;
  } else if (category.value === "jewelry") {
    extraFields.innerHTML = `
      <label>Jewelry Type:</label>
      <select id="jewelryType">
        <option>Necklace</option>
        <option>Earrings</option>
        <option>Bangles</option>
        <option>Ring</option>
      </select>
    `;
  }
});
```

Validation Module:-

Before planning, the app checks the inputs. If no category is chosen, it shows “Please select a category.” If the budget is empty, zero or negative, it shows “Please enter a valid budget.” This prevents wrong results.

```
if (!selectedCategory) {
  result.innerHTML = "⚠️ Please select a category.";
  return;
}

if (!budget || budget <= 0) {
  result.innerHTML = "⚠️ Please enter a valid budget.";
  return;
}
```

Budget Calculation Module:-

Each category has a fixed list of items with percentage shares that add up to 100%. The

Edit with WPS Office

amount for each item is calculated as budget × percentage ÷ 100 and rounded to the nearest rupee.

Category	Share (%)	Items
Home Interior	40 / 20 / 25 / 15	Furniture / Lighting / Decor / Other Expenses
Party Planning	40 / 25 / 20 / 15	Food / Decoration / Venue / Other Expenses
Jewelry Recommendation	70 / 20 / 10	Main Jewelry / Matching Accessories / Other Expenses

```
items.forEach(function (item) {
  const amount = Math.round(budget * item[1] / 100);

  output += `
    <p><strong>${item[0]}:</strong> ₹${amount.toLocaleString("en-IN")} (${item[1]}%)</p>
  `;
});
```

Response Display Module:-

The result is placed in the result area of the page with a heading, one line per item and a success message “Budget plan created successfully!”. Amounts use the ₹ symbol and Indian digit grouping.

Build Web Interface:-

Description

HTML Form (index.html):

- Header with the app name and tagline
- Category dropdown
- Budget input box
- Extra options area filled by JavaScript
- Plan My Budget button and result area
- Footer with copyright

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>PocketSmart AI</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header>
    <h1>💰 PocketSmart AI</h1>
    <p>Smart Budget Planning with AI</p>
  </header>
```

Edit with WPS Office

```
  <main>
    <section class="planner">
      <h2>Plan Your Budget</h2>

      <label for="category">Choose a Category:</label>
      <select id="category">
        <option value="">-- Select Category --</option>
        <option value="home">Home Interior</option>
        <option value="party">Party Planning</option>
        <option value="jewelry">Jewelry Recommendation</option>
      </select>

      <label for="budget">Enter Your Budget (₹):</label>
      <input type="number" id="budget" placeholder="Enter amount">

      <div id="extraFields"></div>

      <button id="planBtn">Plan My Budget</button>

      <div id="result"></div>
    </section>
  </main>

  <footer>
    <p>© 2026 PocketSmart AI</p>
  </footer>

  <script src="script.js"></script>
</body>
</html>
```

Script (script.js):-

The complete JavaScript that handles the dynamic fields, validation and budget calculation.

```
const category = document.getElementById("category");
const budgetInput = document.getElementById("budget");
const extraFields = document.getElementById("extraFields");
const planBtn = document.getElementById("planBtn");
const result = document.getElementById("result");

// Show extra fields based on category
category.addEventListener("change", function () {
  extraFields.innerHTML = "";

  if (category.value === "home") {
    extraFields.innerHTML = `
      <label>Room Type:</label>
      <select id="roomType">
        <option>Living Room</option>
        <option>Bedroom</option>
        <option>Kitchen</option>
      </select>
    `;
  } else if (category.value === "party") {
    extraFields.innerHTML = `
      <label>Party Type:</label>
      <select id="partyType">
        <option>Birthday Party</option>
        <option>Wedding</option>
        <option>Family Function</option>
      </select>
    `;
  } else if (category.value === "jewelry") {
    extraFields.innerHTML = `
      <label>Jewelry Type:</label>
```

Edit with WPS Office

```
        <select id="jewelryType">
          <option>Necklace</option>
          <option>Earrings</option>
          <option>Bangles</option>
          <option>Ring</option>
        </select>
      `;
    }
});

// Budget planning
planBtn.addEventListener("click", function () {
    const selectedCategory = category.value;
    const budget = Number(budgetInput.value);

    if (!selectedCategory) {
        result.innerHTML = "⚠ Please select a category.";
        return;
    }

    if (!budget || budget <= 0) {
        result.innerHTML = "⚠ Please enter a valid budget.";
        return;
    }

    let items = [];

    if (selectedCategory === "home") {
        items = [
            ["Furniture", 40],
            ["Lighting", 20],
            ["Decor", 25],
            ["Other Expenses", 15]
        ];
    } else if (selectedCategory === "party") {
        items = [
            ["Food", 40],
            ["Decoration", 25],
            ["Venue", 20],
            ["Other Expenses", 15]
        ];
    } else if (selectedCategory === "jewelry") {
        items = [
            ["Main Jewelry", 70],
            ["Matching Accessories", 20],
            ["Other Expenses", 10]
        ];
    }

    let output = "<h3>📊 Your Budget Plan</h3>";

    items.forEach(function (item) {
        const amount = Math.round(budget * item[1] / 100);

        output += `
            <p><strong>${item[0]}:</strong> ₹${amount.toLocaleString("en-IN")} (${item[1]}%)</p>
        `;
    });

    output += "<p>✅ Budget plan created successfully!</p>";

    result.innerHTML = output;
});
```

CSS Styling (style.css):-

- Responsive design that fits phone screens

Edit with WPS Office

- Styled buttons, input areas and result container
- Indigo–violet theme with a rounded card layout

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, sans-serif;
  background: #f3f5ff;
  color: #222;
  line-height: 1.6;
}

header {
  background: linear-gradient(135deg, #4f46e5, #7c3aed);
  color: white;
  text-align: center;
  padding: 30px 15px;
}

header h1 {
  font-size: 28px;
  margin-bottom: 8px;
}

header p {
  font-size: 16px;
}

.planner {
  background: white;
  max-width: 500px;
  margin: 30px auto;
  padding: 25px;
  border-radius: 15px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}

.planner h2 {
  text-align: center;
  color: #4f46e5;
  margin-bottom: 20px;
}

label {
  display: block;
  margin-top: 15px;
  font-weight: bold;
}

select,
input {
  width: 100%;
  padding: 12px;
  margin-top: 8px;
  border: 1px solid #ccc;
  border-radius: 8px;
  font-size: 16px;
}

button {
  width: 100%;
  margin-top: 20px;
  padding: 14px;
  background: #4f46e5;
```

Edit with WPS Office

```
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 17px;
  font-weight: bold;
}

button:active {
  background: #3730a3;
}

#result {
  margin-top: 20px;
  line-height: 1.8;
}

footer {
  text-align: center;
  padding: 20px;
  color: #666;
}
```

Run Locally:-

Keep index.html, style.css and script.js in the same folder. Double-click index.html or open it in Google Chrome. No server or installation is needed because the prototype runs fully in the browser.

Functional Testing:-

No.	Test case	Expected result
1	Click Plan My Budget without choosing a category	Message: Please select a category.
2	Choose a category and leave the budget empty	Message: Please enter a valid budget.
3	Home Interior, ₹10,000, Living Room	Furniture ₹4,000, Lighting ₹2,000, Decor ₹2,500, Other ₹1,500
4	Party Planning, ₹20,000, Birthday Party	Food ₹8,000, Decoration ₹5,000, Venue ₹4,000, Other ₹3,000
5	Jewelry, ₹5,000, Necklace	Main Jewelry ₹3,500, Accessories ₹1,000, Other ₹500

Edit with WPS Office

Output Screens:-

a. Home Interior:

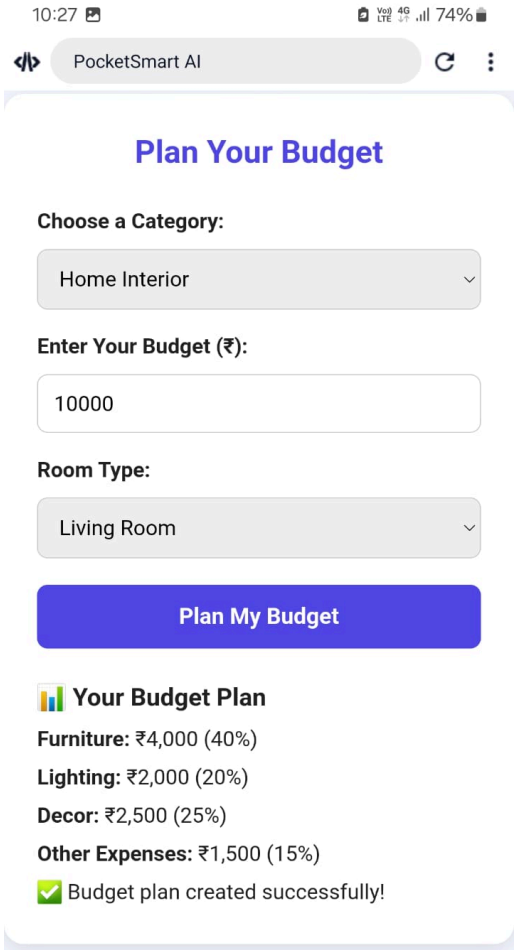


Figure 2: Home Interior budget plan (₹10,000, Living Room)

Edit with WPS Office

b. Party Planning:

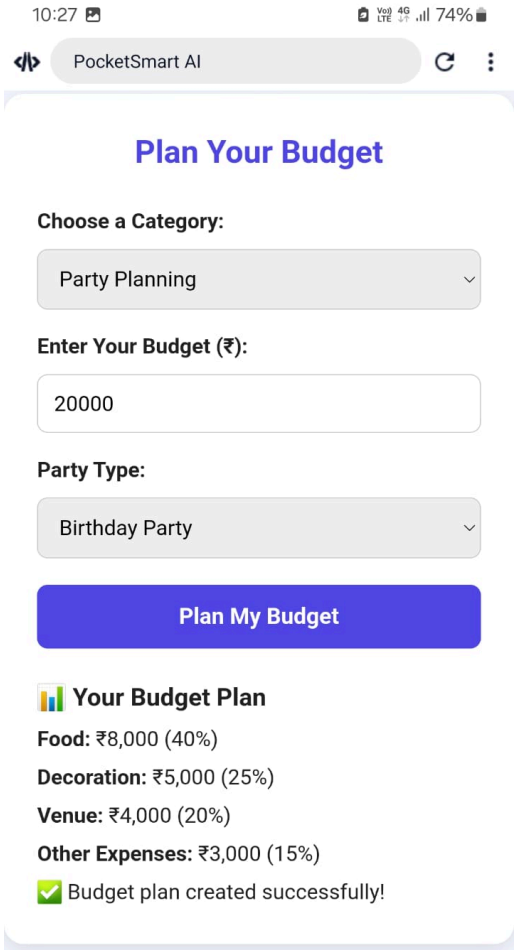


Figure 3: Party Planning budget plan (₹20,000, Birthday Party)

Edit with WPS Office

c. Jewelry Recommendation:

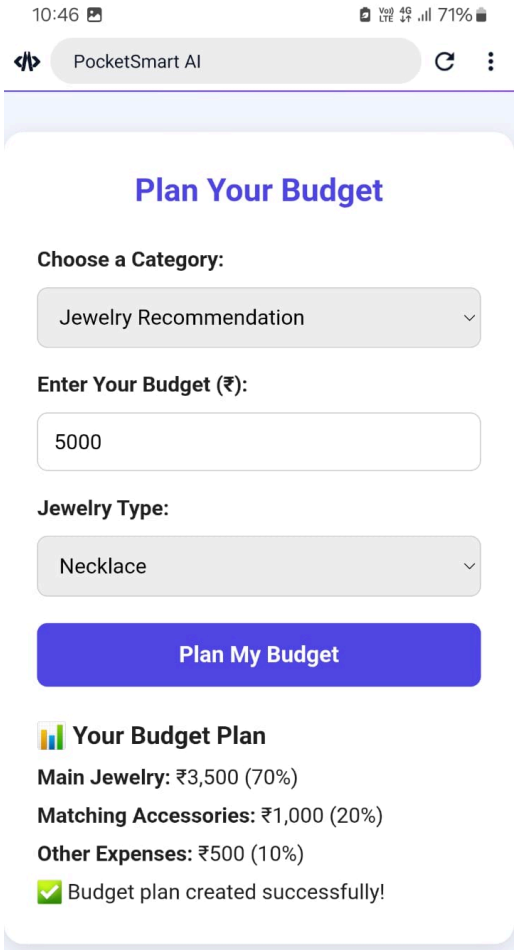


Figure 4: Jewelry budget plan (₹5,000, Necklace)

Implementation Details:-

We started by designing the page layout in HTML with a category dropdown, a budget box and a result area. Then we styled it with CSS to get a clean card layout that works on mobile screens. In JavaScript, we added an event listener to show extra options for each category, wrote validation for the inputs, and defined the percentage split for each category. Finally, we tested the application on a mobile browser with different categories and budget amounts.

Result and Discussion:-

After testing, the project gives correct budget splits for all three categories. In every test the item amounts added up to the entered budget. The interface is very simple, so even non-technical users can use it easily, and the plan appears instantly after clicking the button. It replaces manual calculation and gives users a clear starting point for their spending.

Edit with WPS Office

Conclusion:-

PocketSmart AI is a successful implementation of a simple and useful budget planning tool. It turns a single budget amount into a practical plan and meets its objectives of being fast, clear and user-friendly. This project has been a great learning experience for us in web development, JavaScript logic and cloud architecture design.

Future Enhancement:-

In future, we plan to add:

- Google Gemini based personalised budget splits and recommendations.
- Live product suggestions with prices from shopping platforms.
- User login and saved budget history using Amazon DynamoDB.
- More categories such as travel, weddings and gadgets.
- Mobile application for Android.
- Support for multiple languages including Tamil.

Video link:-

(Add your video link here)

_____end project_____

Edit with WPS Office